

Reviewer Pack — Polymatroid Rank Lower Bound for Monotone k-CLIQUE

Entry aae5f38e2e54 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 21:20:49 UTC

Polymatroid Rank Lower Bound for Monotone k-CLIQUE

Entry ID: aae5f38e2e54 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-10 21:20:49 UTC

1. Conjecture

Field A (mathematical branch): Polymatroid Theory **Field B** (complexity object): Monotone Circuit Size for k-CLIQUE

Statement:

For a k-CLIQUE instance with n vertices, the polymatroid rank of its incidence matrix is $\Omega(n)$ if the instance is a complete graph, and $O(\log n)$ for any instance with a monotone circuit of size $\text{poly}(n)$.

Rationale (proposer's reasoning):

Polymatroid rank captures the interdependency of clauses, which mirrors the combinatorial complexity of monotone circuits. High rank implies structural constraints that may necessitate exponential circuit size.

Taxonomy category: BOUNDED_ARITHMETIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b762dee724f99716

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(matrix):
    n = len(matrix)
    for i in range(n):
        # Find pivot
        max_row = i
        for k in range(i+1, n):
            if abs(matrix[k][i]) > abs(matrix[max_row][i]):
                max_row = k
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

        # Eliminate
        factor = Fraction(1, matrix[i][i])
        for j in range(i, n):
            matrix[i][j] *= factor

        for k in range(n):
            if k != i:
                factor = Fraction(matrix[k][i])
                for j in range(i, n):
                    matrix[k][j] -= factor * matrix[i][j]

def rank_polymatroid(clauses):
    n = len(clauses)
    matrix = [[0] * (n + 1) for _ in range(n)]

    for i in range(n):
        for j in range(i+1, n):
            count = sum(1 for clause in clauses if set(range(len(clause))) & set([i, j]))
            matrix[i][j] = count
            matrix[j][i] = count

    gaussian_elimination(matrix)

    rank = 0
    for i in range(n):
        if any(matrix[i][j] != 0 for j in range(i+1, n)):
            rank += 1

    return rank

def run_trial(seed: int) -> dict:
```

```

random.seed(seed)
n = random.randint(5, 40)
clauses = []

# Generate a random 3-CNF formula
for _ in range(n):
    clause = [random.choice([-1, 1]) * (i + 1) for i in range(random.randint(2, 3))]
    clauses.append(clause)

rank = rank_polymatroid(clauses)

# Determine if the instance is hard or easy based on monotone circuit size
is_hard_instance = any(abs(x) == x for clause in clauses for x in clause)

metric_value = rank
instances_tested = 1

conjecture_holds = False
counterexample = ""

if is_hard_instance:
    if rank >= n / 2: # Arbitrary threshold to consider the instance hard
        conjecture_holds = True
else:
    if rank <= math.log(n, 2):
        conjecture_holds = True

return {
    "metric_name": "polymatroid_rank",
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or list(range(2, 200, 7))[:30]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    std_rank = math.sqrt(sum((r["metric_value"] - mean_rank) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
        counterexample = "mapping_undefined"

```

```
print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_62dc92df.py", line 100, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_62dc92df.py", line 68, in run_trial
    rank = rank_polymatroid(courses)
          ^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_62dc92df.py", line 49, in rank_polymatroid
    gaussian_elimination(matrix)
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_62dc92df.py", line 29, in gaussian_elimination
    factor = Fraction(1, matrix[i][i])
            ^^^^^^^^^^^^^^^^^^^^^
```

```
File "/usr/lib/python3.12/fractions.py", line 281, in __new__
    raise ZeroDivisionError('Fraction(%s, 0)' % numerator)
```

ZeroDivisionError: Fraction(1, 0)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with ZeroDivisionError during Gaussian elimination, preventing result collection. The crash suggests potential issues with matrix construction or singular matrices, but no conclusive data was produced. | next: Debug the Gaussian elimination implementation to handle singular matrices or zero division errors, then re-run tests with controlled examples.

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111253
2	propose	ollama_remote	qwen3:8b	0	0	67586
3	preregistration	ollama_remote	qwen3:8b	0	0	24125
4	novelty	ollama_remote	qwen3:8b	0	0	20666
5	novelty	ollama_remote	qwen3:8b	0	0	14009
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10940

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10228
8	judge	ollama_remote	qwen3:8b	0	0	20719

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 279526 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in research/audit/aae5f38e2e54.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/aae5f38e2e54.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/aae5f38e2e54.tar.gz (if generated)